TP3 : Architecture distribuée

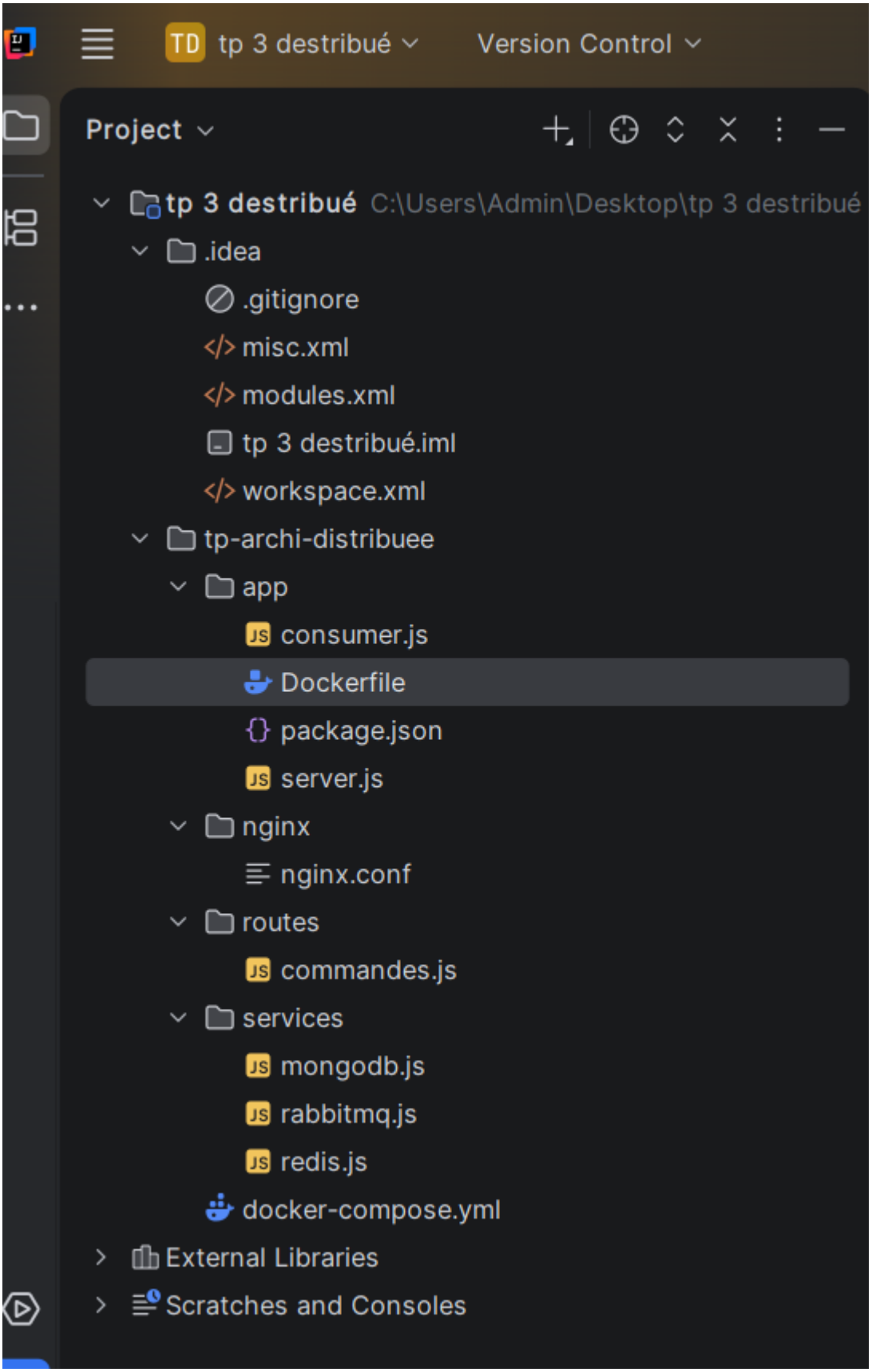
Réalisé par :
Assamri Salma

Encadré par :
Pr.EL Barazoui

Objectifs :

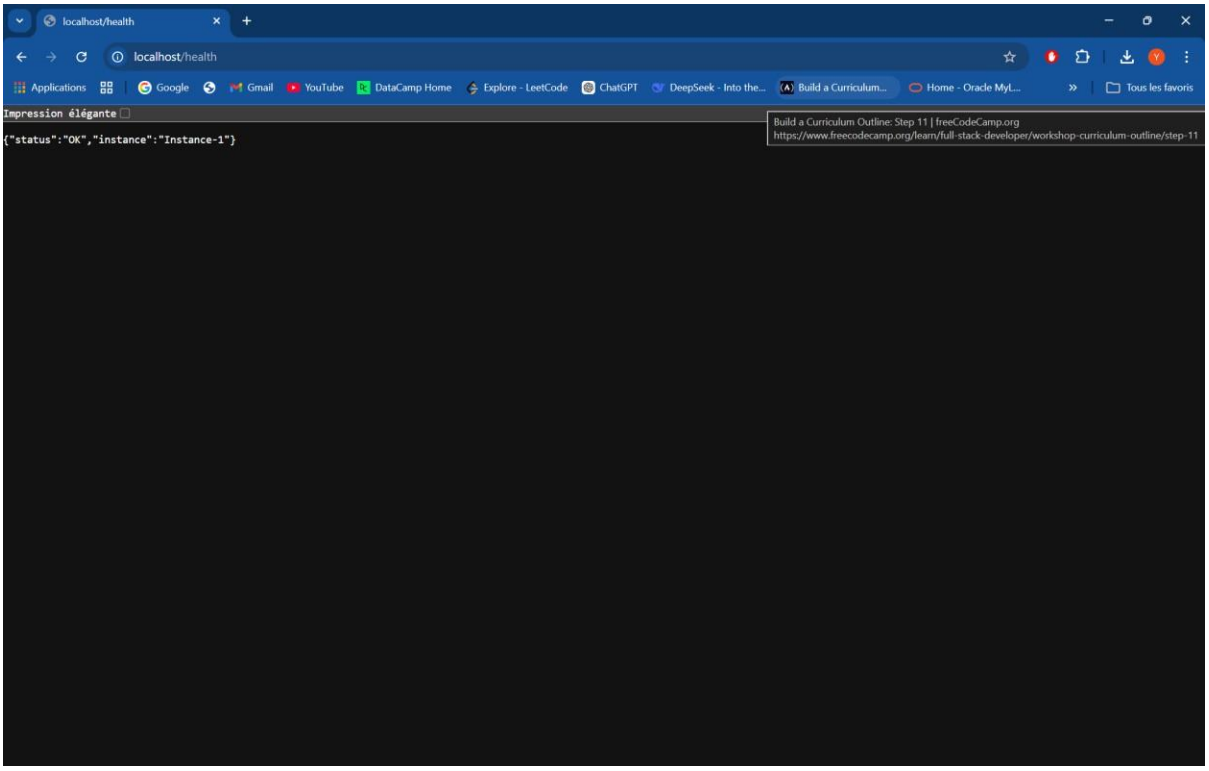
- Déployer une architecture distribuée complète composée de 6 composants interconnectés.
- Comprendre le rôle de chaque composant :
  - Client
  - Load Balancer
  - Serveur applicatif
  - Cache
  - Base de données
  - Bus de messages
- Implémenter un flux complet :
  - POST commande → stockage dans MongoDB
  - Cache Redis pour les requêtes GET
  - Notification via RabbitMQ
- Tester et observer le comportement distribué avec Postman.

Structure :

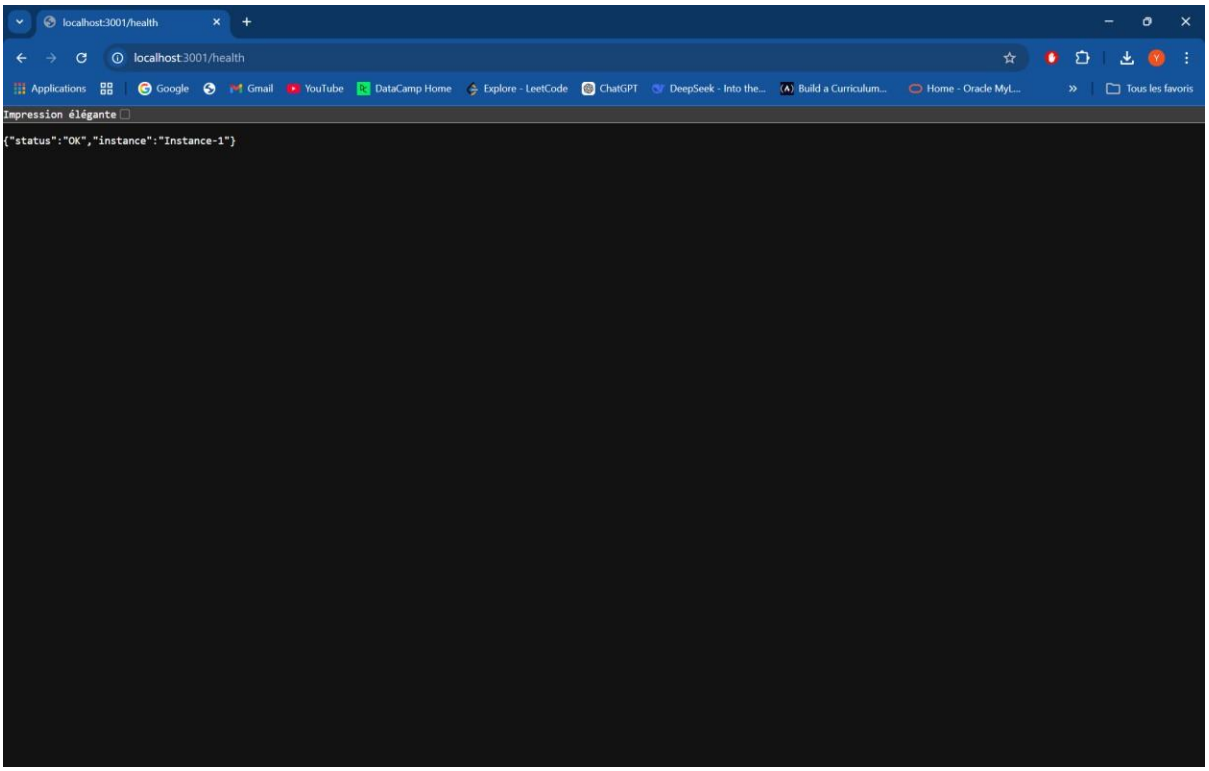


Url à ouvrir :

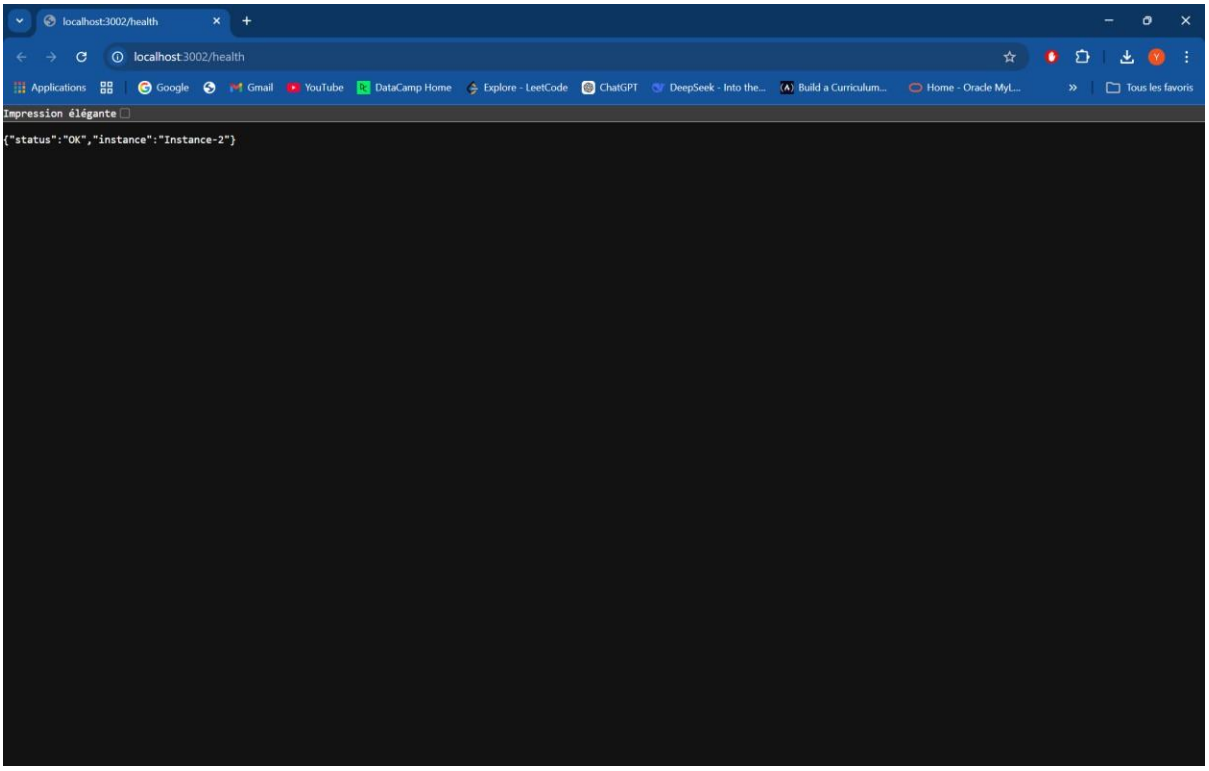
http://localhost/health :



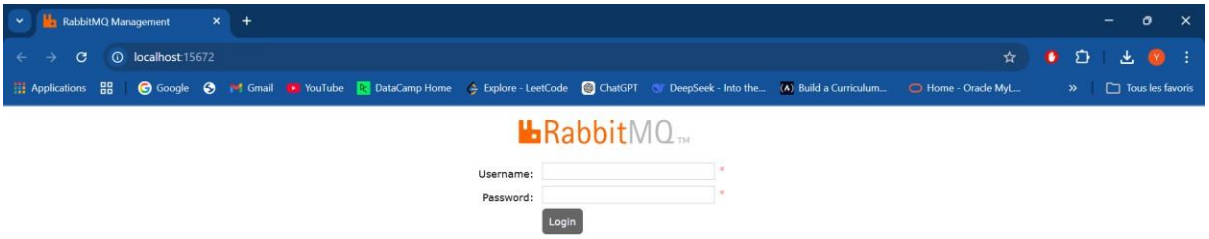
http://localhost:3001/health :



http://localhost:3002/health :

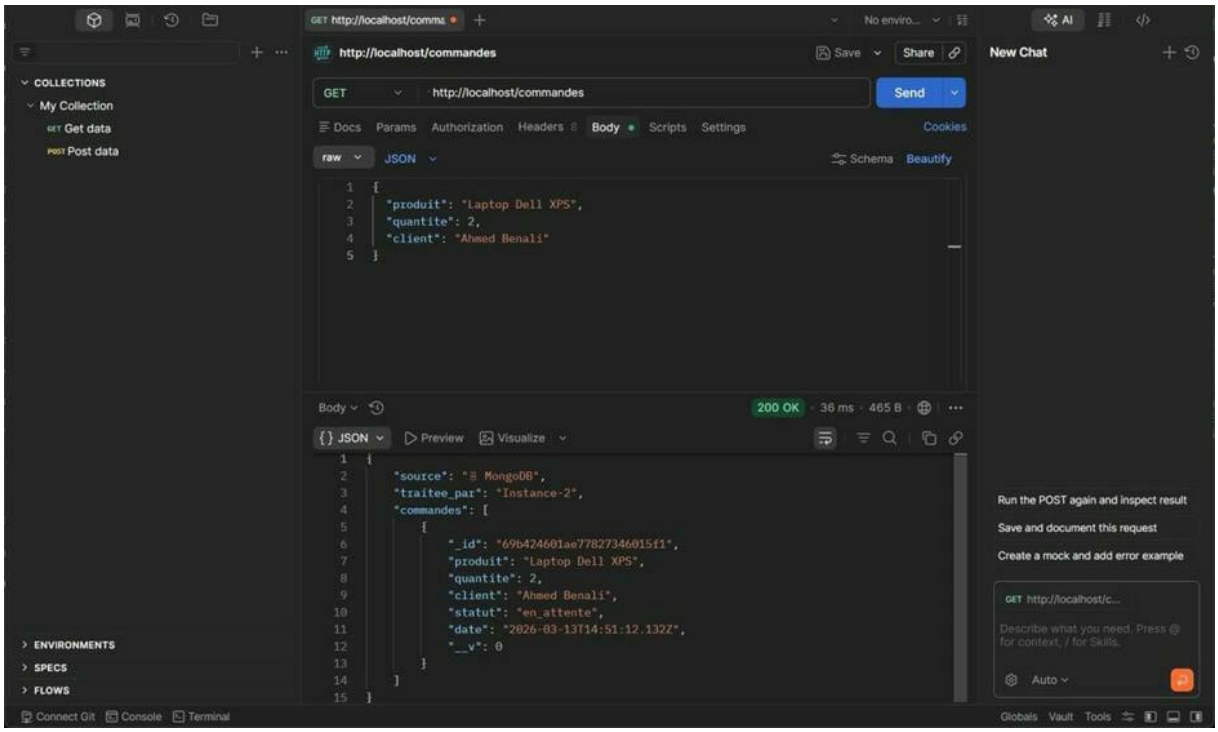


http://localhost:15672 :

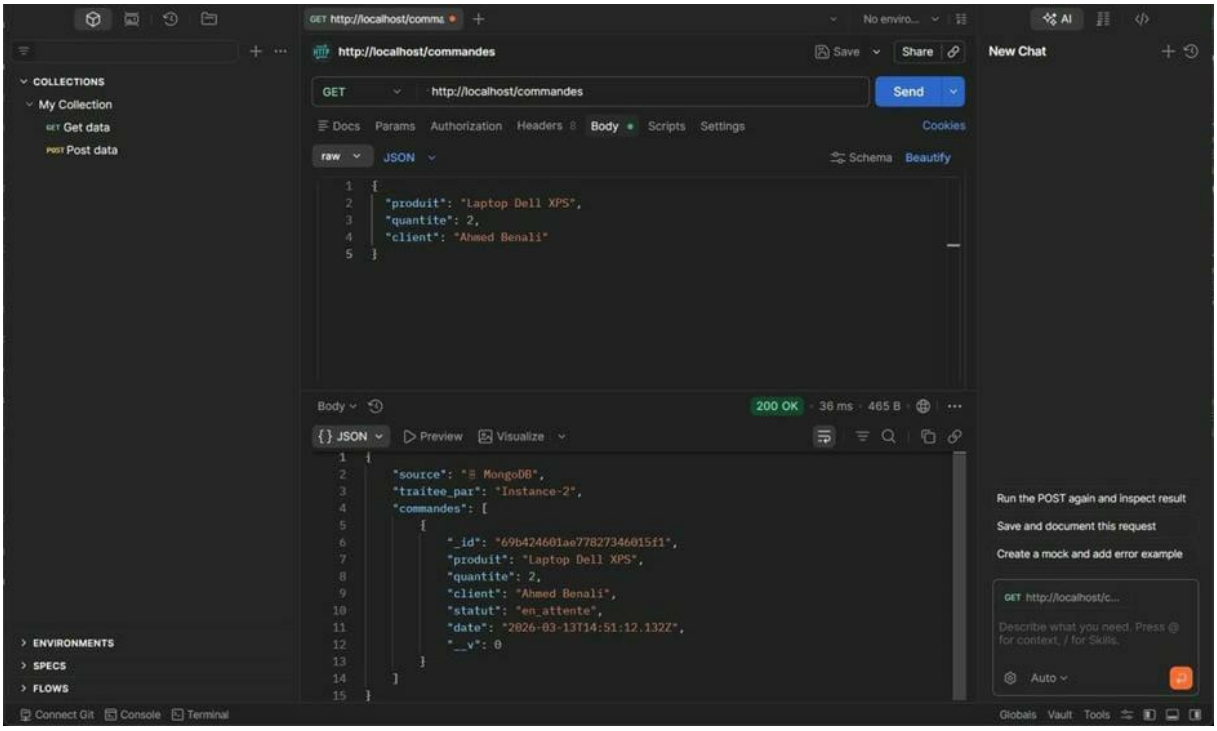


Test avec Postman :

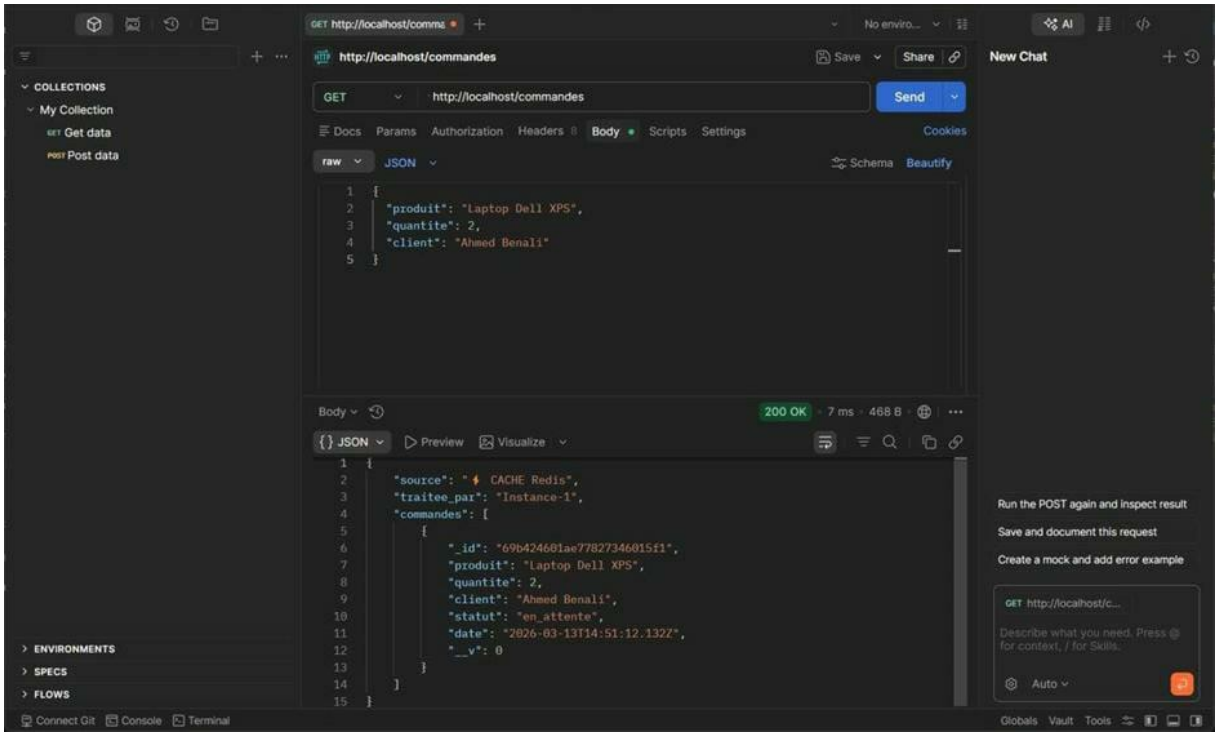
Test 1 :



Test 2 :


GET http://localhost/comma
http://localhost/commandes
Save
Share
New Chat
COLLECTIONS
My Collection
Get data
Post data
GET
http://localhost/commandes
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"produit": "Laptop Dell XPS",
"quantite": 2,
"client": "Ahmed Benali"
}
Body
200 OK
36 ms
465 B
JSON
Preview
Visualize
{
"source": "MongoDB",
"traitee_par": "Instance-2",
"commandes": [
{
"_id": "69b424601ae7782734601 5f1",
"produit": "Laptop Dell XPS",
"quantite": 2,
"client": "Ahmed Benali",
"statut": "en_attente",
"date": "2026-03-13T14:51:12.132Z",
"__v": 0
}
]
}
Run the POST again and inspect result
Save and document this request
Create a mock and add error example
Describe what you need. Press @ for context, / for Skills.
Auto
ENVIRONMENTS
SPECS
FLOWS
Connect Git
Console
Terminal
Globals
Vault
Tools

Test 3 :


GET http://localhost/comma
http://localhost/commandes
Save
Share
New Chat
COLLECTIONS
My Collection
Get data
Post data
GET
http://localhost/commandes
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"produit": "Laptop Dell XPS",
"quantite": 2,
"client": "Ahmed Benali"
}
Body
200 OK
7 ms
468 B
JSON
Preview
Visualize
{
"source": " CACHE Redis",
"traitee_par": "Instance-1",
"commandes": [
{
"_id": "69b424601ae7782734601 5f1",
"produit": "Laptop Dell XPS",
"quantite": 2,
"client": "Ahmed Benali",
"statut": "en_attente",
"date": "2026-03-13T14:51:12.132Z",
"__v": 0
}
]
}
Run the POST again and inspect result
Save and document this request
Create a mock and add error example
Describe what you need. Press @ for context, / for Skills.
Auto
ENVIRONMENTS
SPECS
FLOWS
Connect Git
Console
Terminal
Globals
Vault
Tools

Test 4 :

GET http://localhost/commandes

Send

Body

```
[
  "produit": "Laptop Dell XPS",
  "quantite": 2,
  "client": "Ahmed Benali"
]
```

200 OK · 11 ms · 468 B

```
{
    "source": "⚡ CACHE Redis",
    "traitee_par": "Instance-2",
    "commandes": [
        {
            "_id": "69b424601ae7782734601 5f1",
            "produit": "Laptop Dell XPS",
            "quantite": 2,
            "client": "Ahmed Benali",
            "statut": "en_attente",
            "date": "2026-03-13T14:51:12.132Z",
            "__v": 0
        }
    ]
}
```

GET http://localhost/commandes

Send

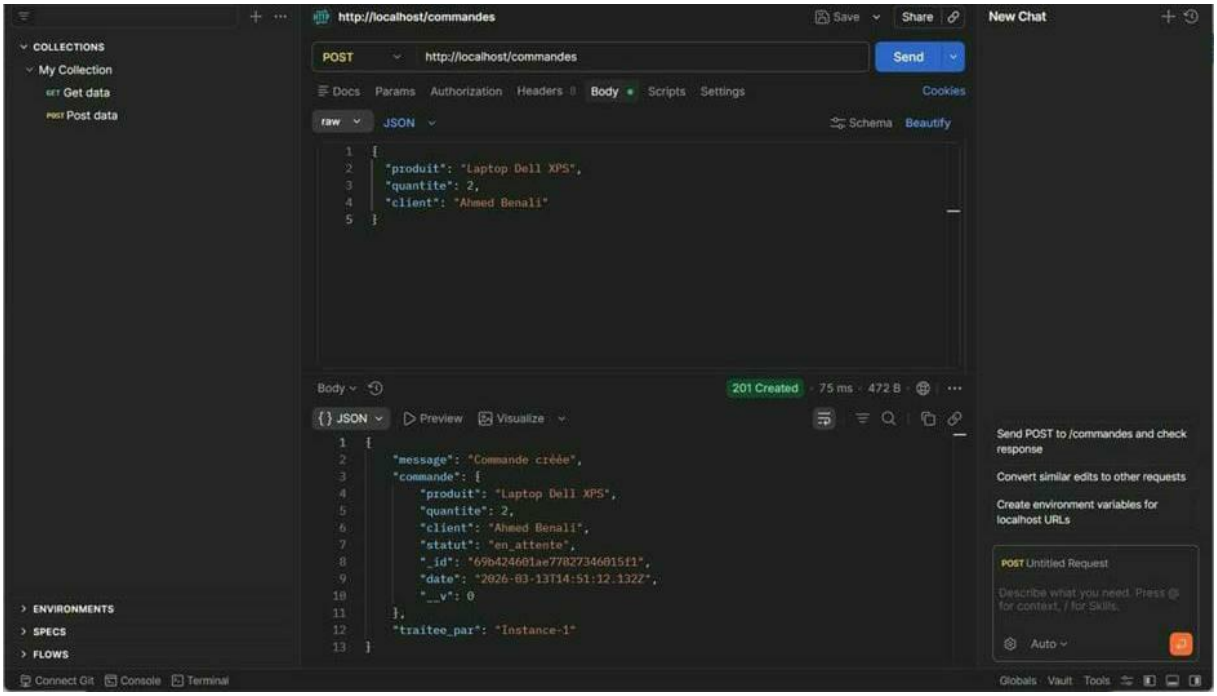
Body

```
[
  "produit": "Laptop Dell XPS",
  "quantite": 2,
  "client": "Ahmed Benali"
]
```

200 OK · 53 ms · 465 B

```
{
    "source": "🗄 MongoDB",
    "traitee_par": "Instance-1",
    "commandes": [
        {
            "_id": "69b424601ae7782734601 5f1",
            "produit": "Laptop Dell XPS",
            "quantite": 2,
            "client": "Ahmed Benali",
            "statut": "en_attente",
            "date": "2026-03-13T14:51:12.132Z",
            "__v": 0
        }
    ]
}
```

Test 5 :



Questions :

Q1- Quel composant reçoit en premier les requêtes du client ? Quel algorithme utilise-t-il ? :

C'est Nginx qui reçoit en premier toutes les requêtes des clients (sur le port 80). Il utilise l'algorithme **Round-robin** par défaut, qui distribue les requêtes en alternance entre les instances : requête 1 → app1, requête 2 → app2, requête 3 → app

Q2- Que se passe-t-il si app1 tombe en panne ? Nginx continue-t-il à fonctionner ? Pourquoi ? :

Lorsque app1 tombe en panne, Nginx détecte automatiquement qu'elle ne répond plus et redirige alors la totalité du trafic vers app2, sans interruption de service. Oui, Nginx continue de fonctionner normalement, car

Nginx retire automatiquement app1 du pool de serveurs actifs jusqu'à son redémarrage

Q3- Expliquez la différence observée entre le 1er GET (source MongoDB) et le 2ème GET (source Redis) :

Lors du premier appel GET, le cache Redis est vide, donc Node.js interroge directement MongoDB pour récupérer les commandes. Le résultat est ensuite stocké dans Redis avec une durée de vie de 30 secondes. Lors du deuxième appel GET (dans les 30 secondes), Node.js trouve les données directement dans **Redis** sans interroger MongoDB. La réponse est quasi instantanée. Cela améliore considérablement les performances du système

Q4- Pourquoi invalide-t-on le cache Redis après un POST /commande :

Après chaque création de commande, les données stockées dans le cache Redis deviennent **obsolètes** car elles ne contiennent pas la nouvelle commande. Si on ne supprime pas le cache, le prochain appel GET retournerait des données incomplètes et incorrectes. L'instruction redis.del('toutes_commandes') force donc le prochain GET à aller chercher les données à jour directement dans MongoDB, garantissant ainsi la **cohérence des données**

Q5- En quoi le bus de messages RabbitMQ améliore-t-il la résilience du système ? :

RabbitMQ améliore la résilience grâce au découplage entre le serveur Node.js et le service de notification, le serveur Node.js n'attend pas la confirmation de la notification avant de répondre au client, ce qui évite tout blocage. La communication est asynchrone, ce qui rend le système plus robuste face aux pannes partielles

Q6- Si on voulait ajouter une 3ème instance Node.js, quelle ligne faudrait-il modifier dans nginx.conf ? :

Il faudrait ajouter la ligne server app3:3003 dans le bloc `upstream` du fichier `nginx/nginx.conf`